

Reverse a Given Linked List

Study Notebook — Brute Force, Iterative Optimal, and Recursive Approaches

Based on the supplied TUF transcript and the user-provided Java ListNode definition/code. The notebook follows the requested sequence: introduction → problem statement → concepts → approaches → pseudocode → transcript core code → complete Java → static visual → complexity → edge cases → revision → conclusion.

Core idea

1 → 3 → 2 → 5 → NULL

After reversal:

5 → 2 → 3 → 1 → NULL

Source basis: the transcript introduces a two-pass stack/data-replacement method, then optimizes by reversing links in one pass, and finally explains a recursive solution. ■filecite■turn5file0■L19-L39■ ■filecite■turn5file0■L107-L179■

1. Introduction

The task is to reverse a singly linked list. The important requirement is to return the **new head** of the reversed list. The transcript first demonstrates an approach that stores values externally, then moves to changing links directly, and finally explains recursion as another implementation. ■filecite■turn5file0■L5-L19■

For a list such as $1 \rightarrow 3 \rightarrow 2 \rightarrow 5$, reversal means the node order becomes $5 \rightarrow 2 \rightarrow 3 \rightarrow 1$. The data values stay with their nodes; the next links are changed in the optimized approach. ■filecite■turn5file0■L199-L227■

2. Problem Statement

Given: the head of a singly linked list.

Task: reverse the linked list.

Return: the new head of the reversed linked list. ■filecite■turn5file0■L7-L19■

Example

Original: $1 \rightarrow 3 \rightarrow 2 \rightarrow 5 \rightarrow \text{NULL}$

Reverse: $5 \rightarrow 2 \rightarrow 3 \rightarrow 1 \rightarrow \text{NULL}$

3. Required Concepts

Node reference: Each node contains data (`val`) and a `next` reference. Reversal is performed by changing these `next` references.

Three-pointer pattern: In the iterative solution, `temp` is the current node, `front` saves the original next node, and `prev` stores the already-reversed part.

Recursion: The recursive solution reduces the problem to a smaller list, reaches the one-node/empty-list base case, then reconnects links while returning from recursion. ■filecite■turn5file0■L599-L671■

4. Approach 1 — Brute Force Using a Stack

The transcript describes the first approach as taking all elements out of the linked list into a stack, then traversing the list again and replacing node data with values popped from the stack. Because a stack is LIFO, the last value becomes the first value written back. ■filecite■turn5file0■L23-L103■

4.1 Algorithm

Pass 1: traverse from `head` to `null` and push every node value into the stack. Pass 2: traverse from `head` again and replace each node value with the stack top, popping after use. The transcript explicitly describes this as a two-pass algorithm. ■filecite■turn5file0■L107-L167■

4.2 Pseudocode

```
temp = head
stack = empty

while temp != null:
    push temp.val into stack
    temp = temp.next

temp = head
while temp != null:
    temp.val = stack.top()
    pop stack
    temp = temp.next

return head
```

4.3 Original Core Algorithm Lines From the Transcript

```
temp = head
stack = empty
while temp != null:
    stack.push(temp.data)
    temp = temp.next

temp = head
while temp != null:
    temp.data = stack.top()
    stack.pop()
    temp = temp.next
```

The transcript describes `stack.push`/`stack.add` and `stack.pop` depending on the language.

■filecite■turn5file0■L119-L167■

4.4 Complete Java Code

```
import java.util.*;

class Solution {
    public ListNode reverseList(ListNode head) {
        Stack<Integer> stack = new Stack<>();
        ListNode temp = head;

        while(temp != null){
            stack.push(temp.val);
            temp = temp.next;
        }

        temp = head;
        while(temp != null){
            temp.val = stack.pop();
            temp = temp.next;
        }

        return head;
    }
}
```

This approach reverses the **data order** rather than physically reversing the links. The transcript highlights this distinction. ■filecite■turn5file0■L95-L105■

Static structure

Before: head
↓
[1] → [3] → [2] → [5] → NULL

Stack after pass 1:
bottom [1, 3, 2, 5] top

After pass 2 (data replaced):
[5] → [2] → [3] → [1] → NULL

4.5 Complexity

Time: $O(n)$, because the list is traversed twice (the transcript expresses this as $O(2n)$). **Auxiliary space:** $O(n)$, because all values are stored in the stack. ■filecite■turn5file0■L167-L185■

5. Approach 2 — Iterative Optimal: Reverse the Links

The optimized approach removes the external stack. Instead of changing values, it changes the `next` pointer of every node so that each link points backward. The transcript identifies this as the way to optimize external space.

■filecite■turn5file0■L187-L227■

5.1 Pointer Roles

- `temp`: current node being processed.
- `front`: saves `temp.next` before the link is reversed; this prevents losing the rest of the list.
- `prev`: previous node; it becomes the new head when the loop ends.

5.2 Pseudocode

```
prev = null
temp = head

while temp != null:
    front = temp.next
    temp.next = prev
    prev = temp
    temp = front

return prev
```

5.3 Original Core Algorithm Lines From the Transcript

```
while temp != null:
    front = temp.next
    temp.next = prev
    prev = temp
    temp = front

return prev
```

The transcript derives the sequence in exactly this order: save the front node, point the current node backward, move `prev` to the current node, then move `temp` to the saved front node. ■filecite■turn5file0■L295-L375■

5.4 Complete Java Code — Corrected User Version

```
class Solution {
    public ListNode reverseList(ListNode head) {
        if(head == null){
            return null;
        }

        ListNode temp = head;
        ListNode prev = null;
        ListNode front = null;

        while(temp != null){
            front = temp.next;    // Save the next node
            temp.next = prev;    // Reverse the current link
            prev = temp;         // Move prev forward
            temp = front;        // Move temp to the saved next node
        }

        return prev;            // prev is the new head
    }
}
```

Important correction: `return front;` is incorrect because when the loop finishes, `temp` and `front` are `null`. `prev` points to the first node of the reversed list, so the method must return `prev`. This matches the transcript, which states that `previous` is the new head after the loop. ■filecite■turn5file0■L495-L509■

Static pointer structure

```

Original:
NULL ← prev      temp → [1] → [3] → [2] → [5] → NULL

After reversal completes:
NULL ← [1] ← [3] ← [2] ← [5] ← prev
                        ↑
                    new head

Returned reference: prev

```

5.5 Why `front` Is Necessary

If `temp.next` is changed before saving the original next node, the connection to the unreversed part can be lost. Therefore the order matters: **save** → **reverse** → **advance prev** → **advance temp**. The transcript explicitly emphasizes storing the front node before replacing the link. ■filecite■turn5file0■L307-L327■

5.6 Complexity

Time: $O(n)$, one pass through the nodes. **Auxiliary space:** $O(1)$, because only pointer variables are used and no external data structure is created. ■filecite■turn5file0■L539-L549■

6. Approach 3 — Recursive Reversal

The transcript explains recursion by reducing a larger list to a smaller problem. For zero or one node, the list is already reversed. Otherwise, recursively reverse the rest of the list, then use the returned new head and the current head's next node to reverse the remaining link. ■filecite■turn5file0■L599-L671■

6.1 Base Case

```

if(head == null || head.next == null){
    return head;
}

```

An empty list or a one-node list needs no reversal. This is the smallest problem used as the stopping condition. ■filecite■turn5file0■L627-L671■

6.2 Pseudocode

```

if head is null OR head.next is null:
    return head

newHead = reverseList(head.next)
front = head.next
front.next = head
head.next = null
return newHead

```

6.3 Original Core Algorithm Lines From the Transcript

```

newHead = reverse(head.next)
front = head.next
front.next = head
head.next = null
return newHead

```

The transcript explains that the recursive call returns the new head of the already-reversed smaller list; the current node is then attached behind it. ■filecite■turn5file0■L967-L1003■

6.4 Complete Java Code

```

class Solution {
    public ListNode reverseList(ListNode head) {
        if(head == null || head.next == null){
            return head;
        }

        ListNode newHead = reverseList(head.next);
        ListNode front = head.next;

```

```

        front.next = head;
        head.next = null;

        return newHead;
    }
}

```

Static recursion structure

Before recursive return:

1 → 3 → 2 → 5 → NULL

↓

reverse(2 → 5)

After smaller problem returns:

5 → 2 → NULL

↑

current head = 3

Reconnect:

5 → 2 → 3 → NULL

Continue returning until 5 is the final new head.

6.5 Complexity

Time: $O(n)$. **Auxiliary space:** $O(n)$ for the recursive call stack. The transcript explicitly identifies this as recursive stack space. ■filecite■turn5file0■L1167-L1197■

7. Edge Cases

- Empty list: `head == null` → return `null` in iterative and recursive solutions.
- One node: `head.next == null` → already reversed; return `head`.
- Two nodes: the second node becomes the new head and the first node points to `null`.
- Multiple nodes: each `next` pointer is reversed exactly once.
- Final pointer: after the iterative loop, `prev` is the new head; `front` is `null`.

Two-node edge case

Original: 1 → 2 → NULL

After: 2 → 1 → NULL
↑
new head (prev)

8. Complexity Comparison

Approach	Time	Auxiliary Space	Main idea
Stack / data replacement	O(n)	O(n)	Store values, then write them back in reverse order
Iterative link reversal	O(n)	O(1)	Reverse `next` links using pointers
Recursive link reversal	O(n)	O(n)	Reverse smaller list, then reconnect during return

The transcript gives O(n) time for all three discussed methods; it distinguishes the stack method by O(n) external space, the iterative method by O(1), and the recursive method by O(n) recursive stack space.

■filecite■turn5file0■L167-L185■ ■filecite■turn5file0■L539-L549■ ■filecite■turn5file0■L1167-L1197■

9. Quick Revision

ITERATIVE – remember: SAVE → REVERSE → MOVE PREV → MOVE TEMP

```
front = temp.next;  
temp.next = prev;  
prev = temp;  
temp = front;
```

```
return prev;
```

Most important interview point: `front` protects the remaining list, `prev` becomes the new head, and `temp` moves through the original remaining nodes.

Common mistake: returning `front` after the loop. At loop termination, `front == null`; return `prev`.

10. Conclusion

The transcript presents three ways to reverse a singly linked list. The stack method uses extra storage and changes node data. The iterative method reverses the links directly with constant auxiliary space. The recursive method also reverses links but uses recursion stack space. For the user-provided Java implementation, the pointer logic is correct; the required final return is `return prev;`. ■filecite■turn5file0■L511-L549■

Study order recommended from the supplied material: understand the stack intuition → master the iterative three-pointer pattern → then learn the recursive version and its base case.